

14 - Classical Processes (CP)

Fondamenti Logici dell'Informatica - Modulo 2

Corso di Laurea Magistrale in Informatica – A.A. 2025-2026

Università di Bologna

sintassi di CP

	π calcolo	CP	
$P, Q ::=$	—	$x \leftrightarrow y$	link
	—	$x \triangleright \{ \}$	errore
	$x[]$	$x[]$	invio segnale
	$x().P$	$x().P$	ricevo segnale
	$x\langle y \rangle.P$	—	invio canale esistente
	—	$x[y](P \mid Q)$	invio canale nuovo
	$x(y).P$	$x(y).P$	ricevo canale
	$x \triangleleft b.P$	$x \triangleleft b.P$	scelgo $b \in \{\text{inl}, \text{inr}\}$
	$x \triangleright \{P, Q\}$	$x \triangleright \{P, Q\}$	offro scelta
	$P \mid Q$	—	composizione parallela
	$(x)P$	—	restrizione
	—	$(x)(P \mid Q)$	cut

$$x\langle y \rangle.P = x[z](z \leftrightarrow y \mid P)$$

nomi liberi

Scriviamo $\text{fn}(P)$ per l'insieme dei nomi che occorrono **liberi** in P :

$$\text{fn}(x \leftrightarrow y) = \{x, y\}$$

$$\text{fn}(x \triangleright \{\}) = \{x\}$$

$$\text{fn}(x[y](P \mid Q)) = (\text{fn}(P) \setminus \{y\}) \cup \text{fn}(Q)$$

$$\text{fn}((x)(P \mid Q)) = (\text{fn}(P) \cup \text{fn}(Q)) \setminus \{x\}$$

Scriviamo $\text{bn}(P)$ per l'insieme dei nomi **legati** in P

semantica operativa di CP

Riduzioni di base

- operazioni complementari possono **interagire** (se sono “vicine”)

$$(x)(x \leftrightarrow y \mid P) \rightarrow P\{y/x\}$$

$$(x)(x[] \mid x().P) \rightarrow P$$

$$(x)(x \triangleleft \text{inl}.P \mid x \triangleright \{Q, R\}) \rightarrow (x)(P \mid Q)$$

$$(x)(x[y](P \mid Q) \mid x(z).R) \rightarrow (y)(P \mid (x)(Q \mid R\{y/z\}))$$

semantica operativa di CP

Riduzioni di base

- operazioni complementari possono **interagire** (se sono “vicine”)

$$(x)(x \leftrightarrow y \mid P) \rightarrow P\{y/x\}$$

$$(x)(x[] \mid x().P) \rightarrow P$$

$$(x)(x \triangleleft \text{inl}.P \mid x \triangleright \{Q, R\}) \rightarrow (x)(P \mid Q)$$

$$(x)(x[y](P \mid Q) \mid x(z).R) \rightarrow (y)(P \mid (x)(Q \mid R\{y/z\}))$$

Chiusura per contesti delle riduzioni

$$(x)(P \mid Q) \rightarrow (x)(P' \mid Q) \quad \text{se } P \rightarrow P'$$

- omettiamo la regola simmetrica poiché il parallelo è commutativo

congruenza strutturale

Usiamo il simbolo $\equiv$ per indicare la più piccola **congruenza** tale che:

$$x \leftrightarrow y \equiv y \leftrightarrow x$$

$$(x)(P \mid Q) \equiv (x)(Q \mid P)$$

$$(x)(P \mid (y)(Q \mid R)) \equiv (y)((x)(P \mid Q) \mid R) \quad x \notin \text{fn}(R), y \notin \text{fn}(P)$$

Chiudiamo la riduzione rispetto alla congruenza

$$P \rightarrow Q \quad \text{se } P \equiv R \text{ e } R \rightarrow Q$$

corrispondenza Curry-Howard per CP

CP	logica lineare
tipi	proposizioni
processi	prove
congruenza	equivalenza tra prove
riduzione	semplificazione di prove

Dai sequenti ai giudizi di tipo

$$\vdash A_1, \dots, A_n \quad \Rightarrow \quad P \vdash x_1 : A_1, \dots, x_n : A_n$$

- un **contesto di tipo** è una mappa finita da canali a tipi
- usiamo ancora Γ e Δ per denotare contesti di tipo
- scriviamo $x : A$ per il contesto singoletto che associa A ad x
- scriviamo Γ, Δ per l'**unione disgiunta** di Γ e Δ
- Γ, Δ è definito solo se $\text{dom}(\Gamma) \cap \text{dom}(\Delta) = \emptyset$

regole di tipo

attenzione ai vincoli nascosti

regole di tipo

attenzione ai vincoli nascosti

Costanti

$$\frac{}{x[] \vdash x : \mathbf{1}} \quad \frac{}{x \triangleright \{\} \vdash \Gamma, x : \top} \quad \frac{P \vdash \Gamma}{x().P \vdash \Gamma, x : \perp}$$

Moltiplicativi

$$\frac{P \vdash \Gamma, y : A \quad Q \vdash \Delta, x : B}{x[y](P \mid Q) \vdash \Gamma, \Delta, x : A \otimes B} \quad \frac{P \vdash \Gamma, y : A, x : B}{x(y).P \vdash \Gamma, x : A \wp B}$$

Additivi

$$\frac{P \vdash \Gamma, x : A \quad Q \vdash \Gamma, x : B}{x \triangleright \{P, Q\} \vdash \Gamma, x : A \& B} \quad \frac{P \vdash \Gamma, x : A}{x \triangleleft \text{inl}.P \vdash \Gamma, x : A \oplus B} \quad \dots$$

Altre regole

$$\frac{}{x \leftrightarrow y \vdash x : A, y : A^\perp} \text{ax} \quad \frac{P \vdash \Gamma, x : A \quad Q \vdash \Delta, x : A^\perp}{(x)(P \mid Q) \vdash \Gamma, \Delta} \text{cut}$$

esempi

$$\text{True}(x) = x \triangleleft \text{inl}.x[]$$

$$\text{Not}(x, y) = x \triangleright \{x().\text{False}\langle y \rangle, x().\text{True}\langle y \rangle\}$$

$$\frac{\frac{\frac{}{x[] \vdash x : \mathbf{1}}{\quad} \mathbf{1}}{x[] \vdash x : \mathbf{1}}}{x \triangleleft \text{inl}.x[] \vdash x : \mathbf{1} \oplus \mathbf{1}} \oplus = \frac{}{\text{True}\langle x \rangle \vdash x : \mathbf{1} \oplus \mathbf{1}}$$

esempi

$$\text{True}(x) = x \triangleleft \text{inl}.x[]$$

$$\text{Not}(x, y) = x \triangleright \{x().\text{False}\langle y \rangle, x().\text{True}\langle y \rangle\}$$

$$\frac{\frac{\frac{}{x[] \vdash x : \mathbf{1}}{\quad} \oplus}{x \triangleleft \text{inl}.x[] \vdash x : \mathbf{1} \oplus \mathbf{1}}}{\text{True}\langle x \rangle \vdash x : \mathbf{1} \oplus \mathbf{1}} =$$

$$\frac{\frac{\frac{}{x[] \vdash x : \mathbf{1}}{\quad} \oplus}{x \triangleleft \text{inr}.x[] \vdash x : \mathbf{1} \oplus \mathbf{1}}}{\text{False}\langle x \rangle \vdash x : \mathbf{1} \oplus \mathbf{1}} =$$

esempi

$$\text{True}(x) = x \triangleleft \text{inl}.x[]$$

$$\text{Not}(x, y) = x \triangleright \{x().\text{False}\langle y \rangle, x().\text{True}\langle y \rangle\}$$

$$\begin{array}{c}
 \frac{}{x[] \vdash x : \mathbf{1}} \oplus \\
 \frac{}{x \triangleleft \text{inl}.x[] \vdash x : \mathbf{1} \oplus \mathbf{1}} \oplus \\
 \frac{}{\text{True}\langle x \rangle \vdash x : \mathbf{1} \oplus \mathbf{1}} = \\
 \vdots \\
 \frac{}{\text{False}\langle y \rangle \vdash y : B} \perp \\
 \frac{}{x().\text{False}\langle y \rangle \vdash x : \perp, y : B} \perp \\
 \frac{}{x().\text{True}\langle y \rangle \vdash x : \perp, y : B} \perp \\
 \frac{}{x \triangleright \{x().\text{False}\langle y \rangle, x().\text{True}\langle y \rangle\} \vdash x : B^\perp, y : B} \& \\
 \frac{}{\text{Not}\langle x, y \rangle \vdash x : B^\perp, y : B} =
 \end{array}$$

esempi

$$\text{Copy}(x, y) = (z)(\text{Not}\langle x, z \rangle \mid \text{Not}\langle z, y \rangle)$$

$$\frac{\frac{\vdots}{\text{Not}\langle x, z \rangle \vdash x : B^\perp, z : B} \quad \frac{\vdots}{\text{Not}\langle z, y \rangle \vdash z : B^\perp, y : B}}{\frac{(z)(\text{Not}\langle x, z \rangle \mid \text{Not}\langle z, y \rangle) \vdash x : B^\perp, y : B}{\text{Copy}\langle x, y \rangle \vdash x : B^\perp, y : B}} \text{cut} =$$

i comportamenti indesiderati sono impediti

Questi processi non sono sintatticamente corretti

- $(x)(x().P)$ starvation
- $(x)(x[])$ messaggio orfano
- $x \triangleleft b.P \mid x(y).Q$ errore di comunicazione
- $x \triangleleft \text{inl}.P \mid x \triangleleft \text{inr}.Q \mid x \triangleright \{R_1, R_2\}$ race in scrittura
- $x \triangleleft b.P \mid x \triangleright \{Q_1, Q_2\} \mid x \triangleright \{R_1, R_2\}$ race in lettura
- $(x)(y)(x().y[] \mid y().x[])$ deadlock

i comportamenti indesiderati sono impediti

Questi processi non sono sintatticamente corretti

- $(x)(x().P)$ starvation
- $(x)(x[])$ messaggio orfano
- $x \triangleleft b.P \mid x(y).Q$ errore di comunicazione
- $x \triangleleft \text{inl}.P \mid x \triangleleft \text{inr}.Q \mid x \triangleright \{R_1, R_2\}$ race in scrittura
- $x \triangleleft b.P \mid x \triangleright \{Q_1, Q_2\} \mid x \triangleright \{R_1, R_2\}$ race in lettura
- $(x)(y)(x().y[] \mid y().x[])$ deadlock

Gli errori di comunicazione non sono possibili

$$(x)(x \triangleleft b.P \mid x(y).Q)$$

- da cut deduciamo $x \triangleleft b.P \vdash \Gamma, x : A$ e $x(y).Q \vdash \Delta, x : B$ e $A = B^\perp$
- da $\oplus$ deduciamo $A = A_1 \oplus A_2$ e da $\wp$ deduciamo $B = B_1 \wp B_2$
- $\oplus$ **non** è il duale di $\wp$

perché la linearità è importante?

Sistema di tipi tradizionale (o “strutturale”)

- il tipo assegnato a una variabile descrive una **proprietà invariante**
- usare **due o più volte** la stessa variabile **non è un problema**

$$\frac{\begin{array}{c} \Gamma, x : t \vdash E_1 : s_1 \\ \vdots \end{array} \quad \begin{array}{c} \Gamma, x : t \vdash E_2 : s_2 \\ \vdots \end{array}}{\Gamma, x : t \vdash \dots E_1 \dots E_2 \dots : s}$$

perché la linearità è importante?

Sistema di tipi tradizionale (o “strutturale”)

- il tipo assegnato a una variabile descrive una **proprietà invariante**
- usare **due o più volte** la stessa variabile **non è un problema**

$$\begin{array}{c} \Gamma, x : t \vdash E_1 : s_1 \\ \vdots \end{array} \quad \begin{array}{c} \Gamma, x : t \vdash E_2 : s_2 \\ \vdots \end{array}$$

$$\frac{}{\Gamma, x : t \vdash \dots E_1 \dots E_2 \dots : s}$$

Sistema di tipi lineare (o “sub-strutturale”)

- il tipo assegnato a una variabile descrive un **protocollo**
- usare **due o più volte** la stessa variabile **viola il protocollo**

$$\begin{array}{c} P_1 \vdash \Gamma, x : A \\ \vdots \end{array} \quad \begin{array}{c} P_2 \vdash \Gamma, x : A \\ \vdots \end{array}$$

$$\frac{}{\dots P_1 \dots \mid \dots P_2 \dots \vdash \Gamma, x : A}$$

preservazione del tipaggio rispetto a $\equiv$

Lemma

Se $P \equiv Q$ allora $P \vdash \Gamma$ implica $Q \vdash \Gamma$.

Dimostrazione.

Per induzione sulla prova di $P \equiv Q$. Mostriamo solo due casi base, gli altri sono analoghi.

Caso $P = x \leftrightarrow y \equiv y \leftrightarrow x = Q$. Dalla regola ax e usando il fatto che $\cdot^\perp$ è un'involuzione deduciamo $\Gamma = x : A, y : A^\perp = y : A^\perp, x : A^{\perp\perp}$.

Concludiamo con una applicazione della regola ax.

Caso $P = (x)(P_1 \mid P_2) \equiv (x)(P_2 \mid P_1) = Q$. Dalla regola cut deduciamo $\Gamma = \Gamma_1, \Gamma_2$ e $P_i \vdash \Gamma_i, x : A_i$ e $A_1 = A_2^\perp$, ovvero $A_2^\perp = A_1^{\perp\perp}$. Concludiamo con una applicazione della regola cut. □

preservazione del tipaggio rispetto a $\rightarrow$

Teorema

Se $P \rightarrow Q$ allora $P \vdash \Gamma$ implica $Q \vdash \Gamma$.

Dimostrazione.

Per induzione sulla derivazione di $P \rightarrow Q$. Mostriamo solo due casi, gli altri sono lasciati come esercizio.

Caso $P = (x)(x \leftrightarrow y \mid R) \rightarrow R\{y/x\} = Q$. Dalla regola cut deduciamo $\Gamma = \Gamma_1, \Gamma_2$ e $x \leftrightarrow y \vdash \Gamma_1, x : A$ e $R \vdash \Gamma_2, x : A^\perp$. Dalla regola ax deduciamo $\Gamma_1 = y : A^\perp$. Una semplice induzione ci permette di derivare $Q \vdash \Gamma_2, y : A^\perp$ (notare che $y \notin \text{dom}(\Gamma_2)$), ma allora abbiamo $Q \vdash \Gamma$.

Caso $P = (x)(x[] \mid x().Q) \rightarrow Q$. Dalla regola cut deduciamo $\Gamma = \Gamma_1, \Gamma_2$ e $x[] \vdash \Gamma_1, x : A$ e $x().Q \vdash \Gamma_2, x : A^\perp$. Dalla regola per **1** deduciamo che Γ_1 è vuoto e $A = \mathbf{1}$. Dalla regola per $\perp$ deduciamo $Q \vdash \Gamma_2$. Concludiamo osservando che $\Gamma = \Gamma_2$. □

thread

nozione ausiliaria per la prova di progresso

Chiamiamo **x-thread** ogni processo che inizia con un'azione su x

- $x[]$ è un x -thread
- $x().P$ è un x -thread
- $x(y).P$ è un x -thread (ma non un y -thread)
- $x \leftrightarrow y$ è sia un x -thread che un y -thread
- ...

Note

- ogni processo diverso da un cut è un thread
- un thread può contenere cut
 - $x().(y)(P \mid Q)$ è un thread

contesti di riduzione

nozione ausiliaria per la prova di progresso

Chiamiamo **contesto di riduzione** un termine generato dalla grammatica

$$\mathcal{C}, \mathcal{D} ::= [] \mid (x)(\mathcal{C} \mid P) \mid (x)(P \mid \mathcal{C})$$

- un contesto di riduzione è un processo con un “buco” indicato da $[]$
- scriviamo $\mathcal{C}[P]$ per il processo ottenuto sostituendo il buco in $\mathcal{C}$ con P
- nota: la sostituzione $\mathcal{C}[P]$ può “catturare” variabili libere in P
 - $P = x[]$ x è libera
 - $\mathcal{C} = (x)([] \mid x().Q)$ x è legata
 - $\mathcal{C}[P] = (x)(x[] \mid x().Q)$ x è legata

lemma di prossimità (enunciato)

Lemma

Se valgono

1 $x \in \text{fn}(P) \setminus (\text{fn}(\mathcal{C}) \cup \text{bn}(\mathcal{C}))$

2 $\text{bn}(\mathcal{C}) \cap \text{fn}(Q) = \emptyset$

allora esiste $\mathcal{D}$ tale che

$$(x)(\mathcal{C}[P] \mid Q) \equiv \mathcal{D}[(x)(P \mid Q)]$$

lemma di prossimità (dimostrazione)

Dimostrazione per induzione strutturale su $\mathcal{C}$.

Caso $\mathcal{C} = []$. Concludo prendendo $\mathcal{D} \stackrel{\text{def}}{=} []$ usando la riflessività di $\equiv$.

lemma di prossimità (dimostrazione)

Dimostrazione per induzione strutturale su $\mathcal{C}$.

Caso $\mathcal{C} = []$. Concludo prendendo $\mathcal{D} \stackrel{\text{def}}{=} []$ usando la riflessività di $\equiv$.

Caso $\mathcal{C} = (y)(\mathcal{C}' \mid R)$. Da (1) deduco $x \neq y$ e $x \notin \text{fn}(R)$ e $x \in \text{fn}(P) \setminus (\text{fn}(\mathcal{C}') \cup \text{bn}(\mathcal{C}'))$. Da (2) deduco $y \notin \text{fn}(Q)$ e $\text{bn}(\mathcal{C}') \cap \text{fn}(Q) = \emptyset$. Usando l'ipotesi induttiva deduco che esiste $\mathcal{D}'$ t.c.
 $(x)(\mathcal{C}'[P] \mid Q) \equiv \mathcal{D}'[(x)(P \mid Q)]$. Ottengo

$$\begin{aligned}(x)(\mathcal{C}[P] \mid Q) &= (x)((y)(\mathcal{C}'[P] \mid R) \mid Q) \\ &\equiv (x)(Q \mid (y)(\mathcal{C}'[P] \mid R)) \\ &\equiv (y)((x)(Q \mid \mathcal{C}'[P]) \mid R) && x \notin \text{fn}(R), y \notin \text{fn}(Q) \\ &\equiv (y)((x)(\mathcal{C}'[P] \mid Q) \mid R) \\ &\equiv (y)(\mathcal{D}'[(x)(P \mid Q)] \mid R)\end{aligned}$$

e concludo prendendo $\mathcal{D} \stackrel{\text{def}}{=} (y)(\mathcal{D}' \mid R)$.

lemma di prossimità (dimostrazione)

Dimostrazione per induzione strutturale su $\mathcal{C}$.

Caso $\mathcal{C} = []$. Concludo prendendo $\mathcal{D} \stackrel{\text{def}}{=} []$ usando la riflessività di $\equiv$.

Caso $\mathcal{C} = (y)(\mathcal{C}' \mid R)$. Da (1) deduco $x \neq y$ e $x \notin \text{fn}(R)$ e $x \in \text{fn}(P) \setminus (\text{fn}(\mathcal{C}') \cup \text{bn}(\mathcal{C}'))$. Da (2) deduco $y \notin \text{fn}(Q)$ e $\text{bn}(\mathcal{C}') \cap \text{fn}(Q) = \emptyset$. Usando l'ipotesi induttiva deduco che esiste $\mathcal{D}'$ t.c.
 $(x)(\mathcal{C}'[P] \mid Q) \equiv \mathcal{D}'[(x)(P \mid Q)]$. Ottengo

$$\begin{aligned}(x)(\mathcal{C}[P] \mid Q) &= (x)((y)(\mathcal{C}'[P] \mid R) \mid Q) \\ &\equiv (x)(Q \mid (y)(\mathcal{C}'[P] \mid R)) \\ &\equiv (y)((x)(Q \mid \mathcal{C}'[P]) \mid R) && x \notin \text{fn}(R), y \notin \text{fn}(Q) \\ &\equiv (y)((x)(\mathcal{C}'[P] \mid Q) \mid R) \\ &\equiv (y)(\mathcal{D}'[(x)(P \mid Q)] \mid R)\end{aligned}$$

e concludo prendendo $\mathcal{D} \stackrel{\text{def}}{=} (y)(\mathcal{D}' \mid R)$.

Caso $\mathcal{C} = (y)(R \mid \mathcal{C}')$. Analogo al caso precedente.



progresso (enunciato)

Teorema

Se $P \vdash \Gamma$ allora:

- 1 esiste Q tale che $P \rightarrow Q$, oppure
- 2 esistono C e un x -thread Q tali che $P = C[Q]$ e $x \notin \text{bn}(C)$.

Intuizione

- se ho un cut $(x)(P \mid Q)$ e P e Q iniziano con azioni sul canale x , tali azioni devono essere complementari poiché x ha tipi duali in P e Q
- non posso avere deadlock perché in un cut i processi in parallelo condividono esattamente un canale

$$\frac{P \vdash \Gamma, x : A \quad Q \vdash \Delta, x : A^\perp}{(x)(P \mid Q) \vdash \Gamma, \Delta} \text{cut}$$

progresso (dimostrazione)

Si procede per induzione sulla struttura di P .

Se P è un x -thread concludiamo prendendo $C \stackrel{\text{def}}{=} []$ e $Q \stackrel{\text{def}}{=} P$, osservando che $x \in \text{dom}(\Gamma)$.

Se $P = (x)(P_1 \mid P_2)$, allora dalla regola cut deduciamo che $\Gamma = \Gamma_1, \Gamma_2$ e $P_i \vdash \Gamma_i, x : A_i$ dove $A_1 = A_2^\perp$. Usando l'ipotesi induttiva per ciascun $i = 1, 2$ deduciamo che

- (1) esiste Q_i tale che $P_i \rightarrow Q_i$, oppure
- (2) esistono C_i e un x_i -thread Q_i tali che $P_i = C_i[Q_i]$ e $x_i \in \text{dom}(\Gamma_i, x : A_i)$.

Abbiamo quattro combinazioni di sotto-casi da analizzare:

- (1,*) Concludiamo prendendo $Q \stackrel{\text{def}}{=} (x)(Q_1 \mid P_2)$ e osservando che $P \rightarrow Q$.
- (2,1) Concludiamo prendendo $Q \stackrel{\text{def}}{=} (x)(P_1 \mid Q_2)$ e osservando che $P \rightarrow Q$.

(2,2) Distinguiamo ulteriori sotto-casi:

- $(x_1 \neq x)$ Concludiamo prendendo $\mathcal{C} \stackrel{\text{def}}{=} (x)(\mathcal{C}_1 \mid P_2)$ e $Q \stackrel{\text{def}}{=} Q_1$ osservando che $x_1 \in \text{dom}(\Gamma_1) \subseteq \text{dom}(\Gamma)$.
- $(x_2 \neq x)$ Analogo al caso precedente.
- $(x_1 = x_2 = x)$

$$\begin{aligned}(x)(P_1 \mid P_2) &\equiv \mathcal{D}_1[(x)(Q_1 \mid P_2)] \\ &\equiv \mathcal{D}_1[(x)(P_2 \mid Q_1)] \\ &\equiv \mathcal{D}_1[\mathcal{D}_2[(x)(Q_2 \mid Q_1)]]\end{aligned}$$

dove l'esistenza di $\mathcal{D}_1$ e $\mathcal{D}_2$ è garantita dal lemma di prossimità. Ora $(x)(Q_2 \mid Q_1)$ è un cut in cui sia Q_1 che Q_2 sono x -thread tipati con A_1 e A_2 uno duale dell'altro. Consultando le regole di tipo e le regole di riduzione si deduce che esiste Q' tale che $(x)(Q_2 \mid Q_1) \rightarrow Q'$. Da qui concludiamo prendendo $Q \stackrel{\text{def}}{=} \mathcal{D}_1[\mathcal{D}_2[Q']]$ e osservando che $P \rightarrow Q$.

terminazione

Teorema

Per ogni P , non esiste una sequenza infinita di riduzioni $P \rightarrow P_1 \rightarrow P_2 \rightarrow \dots$.

Dimostrazione.

Sia $|P|$ il naturale definito per induzione sulla struttura di P come segue:

$$|x \leftrightarrow y| = 1$$

$$|x[]| = 1$$

$$|x().P| = 1 + |P|$$

$$\vdots = \vdots$$

$$|(x)(P \mid Q)| = 1 + |P| + |Q|$$

Basta constatare che $P \equiv Q$ implica $|P| = |Q|$ e $P \rightarrow Q$ implica $|Q| < |P|$ (ogni riduzione diminuisce la dimensione del processo). □

esercizi

- 1 Scrivere una derivazione di tipo per il processo $\text{And}\langle x, y, z \rangle$
- 2 Scrivere una derivazione di tipo per il processo $(x)(\text{True}\langle x \rangle \mid (y)(\text{False}\langle x \rangle \mid \text{And}\langle x, y, z \rangle))$
- 3 I processi Alice, Bob e Carol sono ben tipati? Cosa si può dire del processo $(x)((x)(\text{Bob}\langle x, u, v \rangle \mid \text{Alice}\langle x \rangle) \mid \text{Carol}\langle x \rangle)$?
- 4 Dimostrare tutti i casi non discussi del teorema di preservazione del tipaggio rispetto a $\rightarrow$.